



Hacettepe Üniversitesi İstatistik Bölümü İST281 Bilgisayar Programlama Dersi

Özyineli (recursive) fonksiyonlar

- Özyineli fonksiyonlar, kendi içinde, kendini çağıran fonksiyonlardır.
- Özel bir yazılım geliştirme yöntemidir.
- Yığın (stack) veri yapısı ile oluşturulan, matematiksel olarak ifade edilebilen bir yöntemdir.
- Yığın veri yapısında yeni gelen bilgi, bir önceki bilginin sonrasına kaydedilir. İlk gelen bilgi en altta, son gelen bilgi en üstte kalır.
- Yığın dolduğunda veya gerektiğinde yığın boşaltılır. Yığın boşaltılırken son gelen bilgi ilk çıkar (last in first out).
- Bu yöntem kullanılarak yazılan fonksiyonlar özyineli niteliğindedir.

- **Örnek:** Verilen bir tamsayıya kadar olan sayıların toplamını özyineli fonksiyon kullanarak bulalım.
- Fonksiyon özyineli olmasaydı, aşağıdaki gibi bir fonksiyon yazabilirdik:

```
def topla_1_n(n):  
    sonuç=0  
    for i in range(1,n+1):  
        sonuç=sonuç+i  
    return sonuç
```

- Çağırarak için:

```
n=int(input("Bir tamsayı giriniz:"))  
toplam=topla_1_n(n)  
print(toplam)
```

- Çözümü özyineli fonksiyon mantığı ile hesaplamak istersek yazacağımız fonksiyon şöyle olur:

```
def özyineli_topla_1_n(n):  
    if n == 1:  
        return 1  
    else:  
        sonuç= n + özyineli_topla_1_n(n-1)  
        return sonuç
```

Özyineli fonksiyon yazarken bir durma koşulu belirlenmesi gerekir. Fonksiyon durma koşulunu sağladığında artık kendini çağırmayı bırakacak, sonraki işlemleri geriye doğru yapmaya başlayacaktır.

Aynı fonksiyon tekrar çağırılıyor.

Fonksiyonu çağıran deyim:

`toplam=özyineli_topla_1_n(5)`

Olsun. İşlemler sırasıyla şöyle gerçekleşir:

`sonuç= 5 + özyineli_topla_1_n(4)`

`sonuç= 4 + özyineli_topla_1_n(3)`

`sonuç= 3 + özyineli_topla_1_n(2)`

`sonuç= 2 + özyineli_topla_1_n(1)`

Bu aşamada, $n=1$ olduğu için, fonksiyonun durma koşulunu sağlanmış olur.

Fonksiyonu çağıran deyim:

`toplam=özyineli_topla_1_n(5)`

Olsun. İşlemler sırasıyla şöyle gerçekleşir:

`sonuç= 5 + özyineli_topla_1_n(4)`

`sonuç= 4 + özyineli_topla_1_n(3)`

`sonuç= 3 + özyineli_topla_1_n(2)`

`sonuç= 2 + özyineli_topla_1_n(1)`

Bu aşamada, $n=1$ olduğu için, fonksiyonun durma koşulunu sağlanmış olur. Her adımda fonksiyon kaldığı yere döner. Böylece return deyimleri çalışmaya başlar.

Bu adımları, bellekte oluşacak tüm değerleri yerine koyarak yazalım.

özyineli_topla_1_n(5)	# n=5 ile ilk çağırma
5 + özyineli_topla_1_n(4)	# n=4 ile ikinci çağırma
5 + 4 + özyineli_topla_1_n(3)	# n=3 ile üçüncü çağırma
5 + 4 + 3 + özyineli_topla_1_n(2)	# n=2 ile dördüncü çağırma
5 + 4 + 3 + 2 + özyineli_topla_1_n(1)	# n=1 ile beşinci çağırma
5 + 4 + 3 + 2 + 1	# beşinci çağırmadan return 1 ile dönüş
5 + 4 + 3 + 3	# dördüncü çağırmadan return 3 ile dönüş
5 + 4 + 6	# üçüncü çağırmadan return 6 ile dönüş
5 + 10	# ikinci çağırmadan return 10 ile dönüş
15	# birinci çağırmadan return 15 ile dönüş

Şimdi aynı yöntemi kullanarak $n!$ değerini hesaplayan bir özyineli fonksiyon yazalım. Özyineli fonksiyon yazarken sondan başa doğru düşünmek yararlı olur. Hesaplamak istediğimiz değer $n!$ olduğuna göre, bir önceki değer, $n * (n-1)!$ değeridir. Geriye doğru her adımda aynı hesaplama geçerlidir. Her aşamada, değer $* (\text{değer}-1)!$ çarpımı ile çözümü bulabiliriz. Son olarak değer 1 'e eşit olması, durma koşulumuzdur.

Örneğin $4!$ değerini hesaplamak istersek:

$$4! = 4 * 3!$$

$$3! = 3 * 2!$$

$$2! = 2 * 1$$

Biçiminde yazabiliriz. Eşitlikleri sırasıyla yerlerine yazarsak,

$$4! = 4 * 3 * 2 * 1$$

Buna göre bir önceki probleme çok benzer olarak faktöryel hesabı yapan bir özyineli fonksiyon yazabiliriz:

```
def faktoryel(n):  
    if n == 1:  
        return 1  
    else:  
        f= n * faktoryel(n-1)  
        return f
```

Fonksiyonun nasıl çalıştığını izlemek için satır aralarına print() deyimleri yazabiliriz.

```
def faktoryel(n):  
    print("fonksiyon n=",n,"değeriyle çağırıldı")  
    if n == 1:  
        return 1  
    else:  
        f= n * faktoryel(n-1)  
        print("n=",n,"değeri için ara sonuç:",n,"*faktoryel(",n-1,")=",f)  
        return f
```

Fonksiyonu $n=4$ için çağırılım:

$n=4$

$f=faktoryel(n)$

$print(f)$

Program çalıştırıldığında, fonksiyonun her çağırılışında birinci $print()$ deyimi çalışacaktır. Çağrılar bitince ikinci $print()$ deyimi çalışacak ve dönüş değerlerini görüntüleyecektir. Ekran şöyle görünür:

fonksiyon $n= 4$ değeriyle çağırıldı

fonksiyon $n= 3$ değeriyle çağırıldı

fonksiyon $n= 2$ değeriyle çağırıldı

fonksiyon $n= 1$ değeriyle çağırıldı

$n= 2$ değeri için ara sonuç: $2 * faktoryel(1)= 2$

$n= 3$ değeri için ara sonuç: $3 * faktoryel(2)= 6$

$n= 4$ değeri için ara sonuç: $4 * faktoryel(3)= 24$

24

Örnek: Fibonacci serisinin n inci terimini hesaplayan bir özyineli fonksiyon yazın. Fonksiyonu çağırarak 0 ve 1 ile başlayan serinin n terimini oluşturun.

Çözüm:

Matematiksel ifade:

$fbnc(n)=n$, eğer $n=0$ veya $n=1$ ise

$fbnc(n)=fbnc(n-2) + fbnc(n-1)$, diğer durumlar.

$n=7$ ise çıktı: 0 1 1 2 3 5 8

Özyineli fonksiyon:

```
def fbnc(n): # n inci fibonacci sayısını hesaplar
    if n <= 1: # durma koşulu
        return n # 0 başlangıç
    else: # özyineli çağırma
        return (fbnc(n - 1) + fbnc(n - 2))
# Fonksiyonun çağırılması
n=7
for i in range(0,n):
    f=fbnc(i)
    print(f, end=" ")
```

Çıktı:

0 1 1 2 3 5 8

Başka bir çözüm:

iki_önceki = 1 # Fonksiyonda değişeceği için rasgele bir değer

```
def fbnc2(n):
```

```
    global iki_önceki
```

```
    if n==0:
```

```
        iki_önceki=0
```

```
        return iki_önceki
```

```
    elif n<=2:
```

```
        iki_önceki=1
```

```
        return iki_önceki
```

```
    önceki=fbnc2(n-1)
```

```
    şimdiki=önceki+iki_önceki
```

```
    iki_önceki=önceki
```

```
    return şimdiki
```

```
# Çağırın bölüm
```

```
n=15
```

```
for i in range(0,n):
```

```
    print(fbnc2(i), end=" ")
```

Çıktı:

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Soru: Verilen özyineli fonksiyonları inceleyin. Nasıl çalıştıklarını ve çıktılarını bulun.

a)

```
def dene1(n):
```

```
    if n>0:
```

```
        print(n,end=" ")
```

```
        return dene1(n-1)
```

```
    else:
```

```
        return n
```

```
#Çağır
```

```
print(dene1(5))
```

b)

```
def dene2(n):
```

```
    if n>0:
```

```
        print(n,end=" ")
```

```
        return dene2(n-1)
```

```
        print(n,end=" ")
```

```
    else:
```

```
        return n
```

```
#Çağır
```

```
print(dene2(5))
```

c)

```
def dene3(n):  
    if n>0:  
        print(n,end=" ")  
        dene3(n-1)  
#Çağır  
dene3(5)
```


d)

```
def dene4(n):  
    if n>0:  
        print(n,end=" ")  
        dene4(n-1)  
        print(n,end=" ")  
#Çağır  
dene4(4)
```

e)

```
def dene5(n):  
    if n>0:  
        dene5(n-1)  
        print(n,end=" ")  
        dene5(n-1)  
#Çağır  
dene5(5)
```